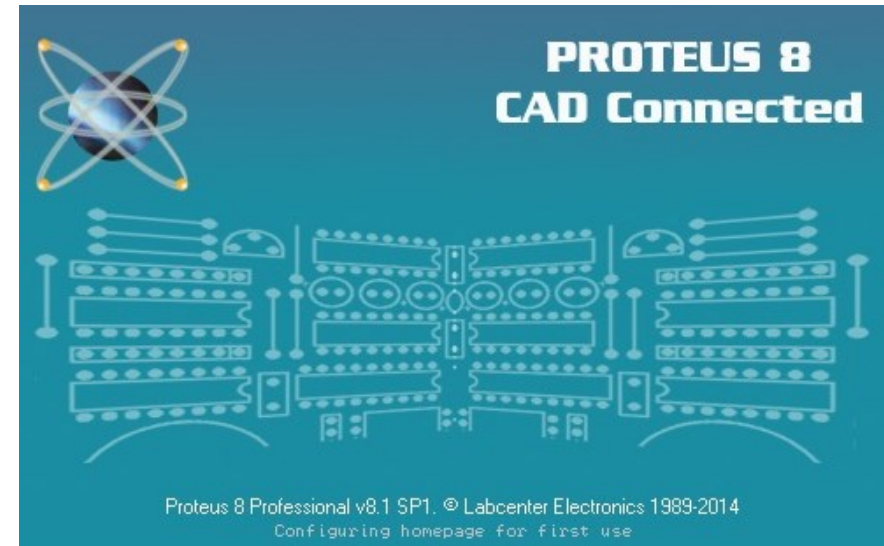




Lab 5: Tìm hiểu và thực hành về GPIO và Timer trên STM32F103C8

GVHD: *Ths. Thân Thế Tùng*

Mô hình mô phỏng



Software Development

SIMULATION

Cấu hình GPIO

Bước 1: Cấp xung nhịp (Clock) cho cổng GPIO

Bước 2: Cấu hình chế độ và tốc độ chân

Bước 3: Thiết lập điện trở kéo (Pull-up/Pull-down) nếu cần

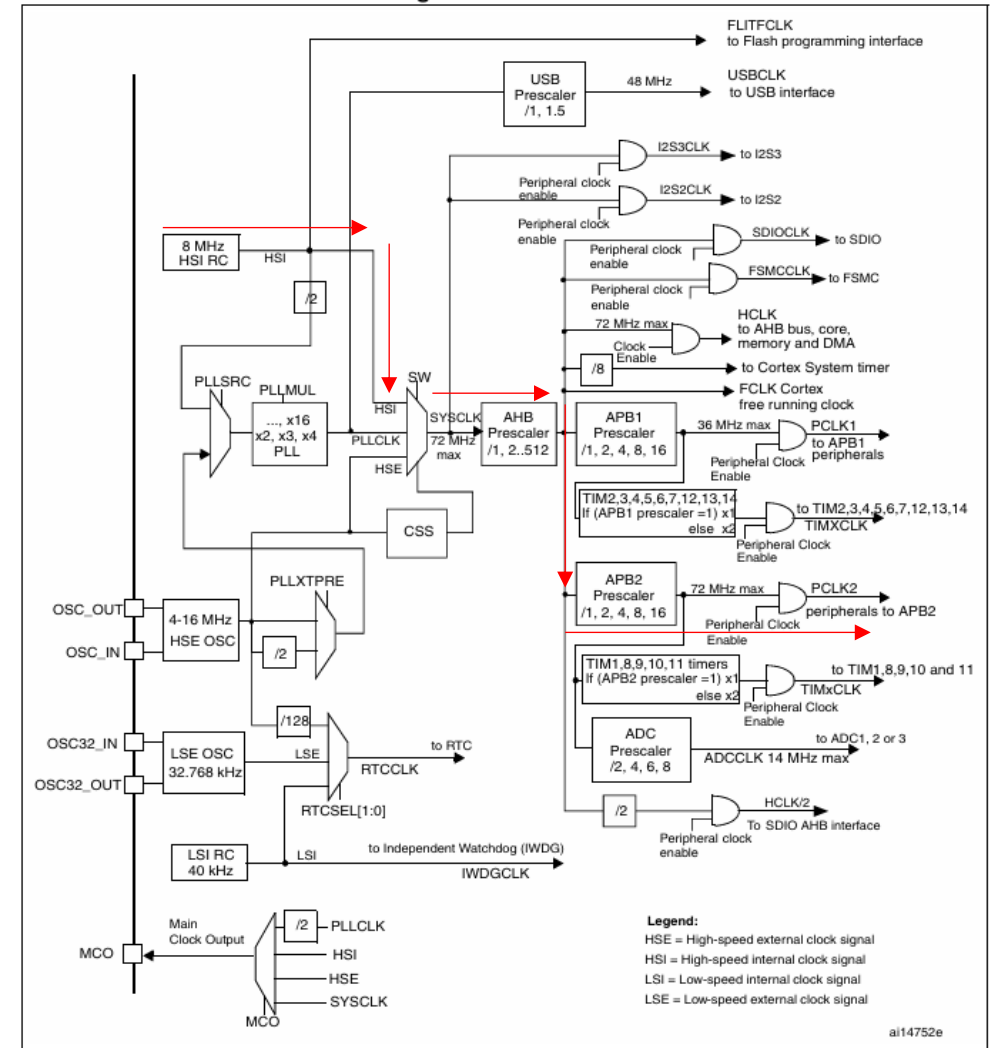
Bước 4: Cấu hình chức năng thay thế (AFIO) và Remap (tùy chọn)

Bước 5: Thao tác dữ liệu trên chân GPIO

Tìm hiểu về Clock

- **HSI (High Speed Internal):** Bộ dao động RC nội 8 MHz. Đây là nguồn mặc định sau khi reset thiết bị
- **HSE (High Speed External):** Bộ dao động ngoài với thạch anh từ 4 đến 16 MHz
- **LSI (Low Speed Internal):** Bộ dao động nội khoảng 40 kHz, dùng cho bộ Watchdog độc lập (IWDG) và có thể làm nguồn cho RTC để thức tỉnh từ chế độ Stop/Standby
- **LSE (Low Speed External):** Thạch anh ngoài 32.768 kHz, chủ yếu dùng cho khối thời gian thực (RTC)
- **PLL (Phase-Locked Loop):** Dùng để nhân tần số từ nguồn HSI (chia 2) hoặc HSE để đạt được tần số hoạt động cao hơn

Figure 8. Clock tree



Cấu hình GPIO – Bước 1 (Cấp xung nhịp (Clock) cho cổng GPIO)

- Tất cả các cổng GPIO (từ Port A đến Port G) đều được kết nối với bus APB2
- Bật xung nhịp cho cổng đó bằng cách thiết lập các bit tương ứng (IOPAEN, IOPBEN,...) trong thanh ghi RCC_APB2ENR

7.3.7 APB2 peripheral clock enable register (RCC_APB2ENR)

Address: 0x18

Reset value: 0x0000 0000

Access: word, half-word and byte access

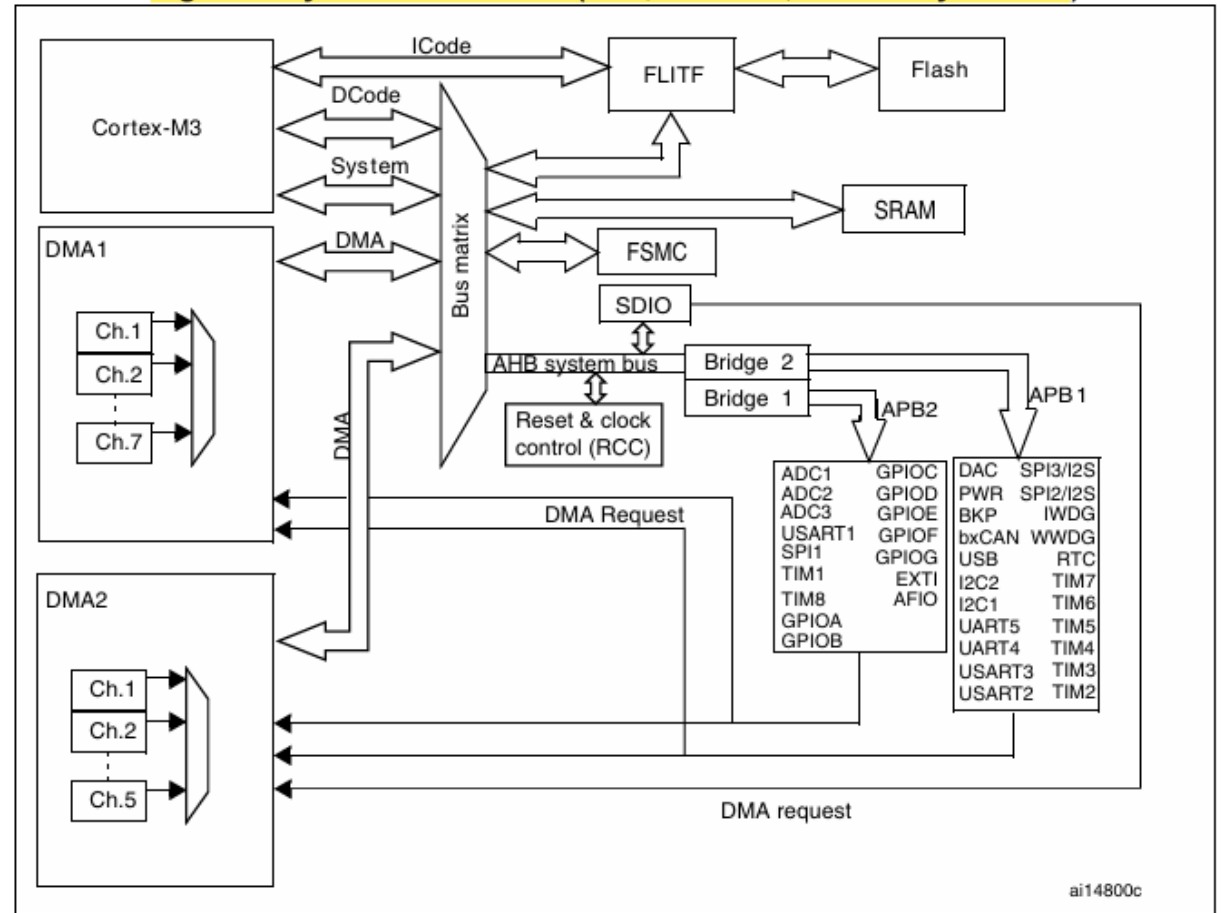
Bit 4 IOPCEN: IO port C clock enable

Set and cleared by software.

0: IO port C clock disabled

1: IO port C clock enabled

Figure 1. System architecture (low-, medium-, XL-density devices)



Cấu hình GPIO – Bước 2 (Cấu hình chế độ và tốc độ chân)

- Mỗi chân GPIO được cấu hình bởi 4 bit trong các thanh ghi cấu hình: **GPIOx_CRL** (cho các chân từ 0 đến 7) và **GPIOx_CRH** (cho các chân từ 8 đến 15)
- **Bit MODE[1:0]:** Xác định hướng và tốc độ tối đa của chân
 - **00:** Chế độ Nhập (Input mode)
 - **01/10/11:** Chế độ Xuất (Output mode) với tốc độ tối đa lần lượt là **10 MHz**, **2 MHz**, và **50 MHz**
- **Bit CNF[1:0]:** Xác định cấu hình cụ thể dựa trên chế độ MODE đã chọn
 - **Trong chế độ Input:** Có thể chọn Analog (00), Floating input (01 - mặc định sau reset), hoặc Input with pull-up/pull-down (10)
 - **Trong chế độ Output:** Có thể chọn General purpose push-pull (00), Open-drain (01), Alternate function push-pull (10), hoặc Alternate function open-drain (11)

Cấu hình GPIO – Bước 2

MODE[1:0]	CNF[1:0]	Chế độ	Kiểu điện	Đặc điểm chính	Ứng dụng
00	00	Input	Analog	Tắt digital buffer, giảm nhiễu	ADC
00	01	Input	Floating	High-Z, dễ nhiễu nếu không có nguồn ngoài	Sensor có driver
00	10	Input	Pull-up / Pull-down	Có điện trở nội, ổn định mức	Button
00	11	Input	Reserved	Không dùng	—
01/10/11	00	Output	Push-pull	Drive mạnh, kéo lên/xuống chủ động	LED, GPIO
01/10/11	01	Output	Open-drain	Chỉ kéo xuống, cần pull-up	I2C, bus chung
01/10/11	10	Output	AF push-pull	Peripheral điều khiển, drive mạnh	UART, SPI, PWM
01/10/11	11	Output	AF open-drain	Peripheral + open-drain	I2C

Cấu hình GPIO – Bước 3

(Thiết lập điện trở kéo (Pull-up/Pull-down) nếu cần)

- Thiết lập điện trở kéo (Pull-up/Pull-down) nếu cần: Nếu chân được cấu hình là Input có điện trở kéo, bạn sử dụng thanh ghi dữ liệu xuất GPIOx_ODR để chọn kiểu kéo
 - Ghi '1' vào bit tương ứng trong ODR để kích hoạt điện trở kéo lên (pull-up)
 - Ghi '0' để kích hoạt điện trở kéo xuống (pull-down)

Cấu hình GPIO – Bước 4 (Cấu hình chức năng thay thế (AFIO) và Remap)

- Nếu sử dụng các chức năng ngoại vi như USART, SPI hoặc Timer trên các chân GPIO, cần bật xung nhịp cho ngoại vi AFIO trong thanh ghi `RCC_APB2ENR`
- Trong trường hợp cần thay đổi vị trí chân mặc định của ngoại vi, phải cấu hình remapping trong thanh ghi `AFIO_MAPR`

Cấu hình GPIO – Bước 5 (Thao tác dữ liệu trên chân GPIO)

- Đọc dữ liệu: Sử dụng thanh ghi GPIOx_IDR (Input Data Register). Đây là thanh ghi chỉ đọc, chứa giá trị logic hiện tại của chân
- Xuất dữ liệu: Có thể ghi trực tiếp vào thanh ghi GPIOx_ODR

Cấu hình GPIO – Ví dụ

```
#define RCC_APB2ENR (*(volatile unsigned int*)0x40021018)
#define GPIOC_CRH (*(volatile unsigned int*)0x40011004)
#define GPIOC_ODR (*(volatile unsigned int*)0x4001100C)
```

```
void delay()
{
    for(int i=0;i<50000;i++);
}
```

```
int main()
{
    /* clock GPIOC */
    RCC_APB2ENR |= (1<<4);
```

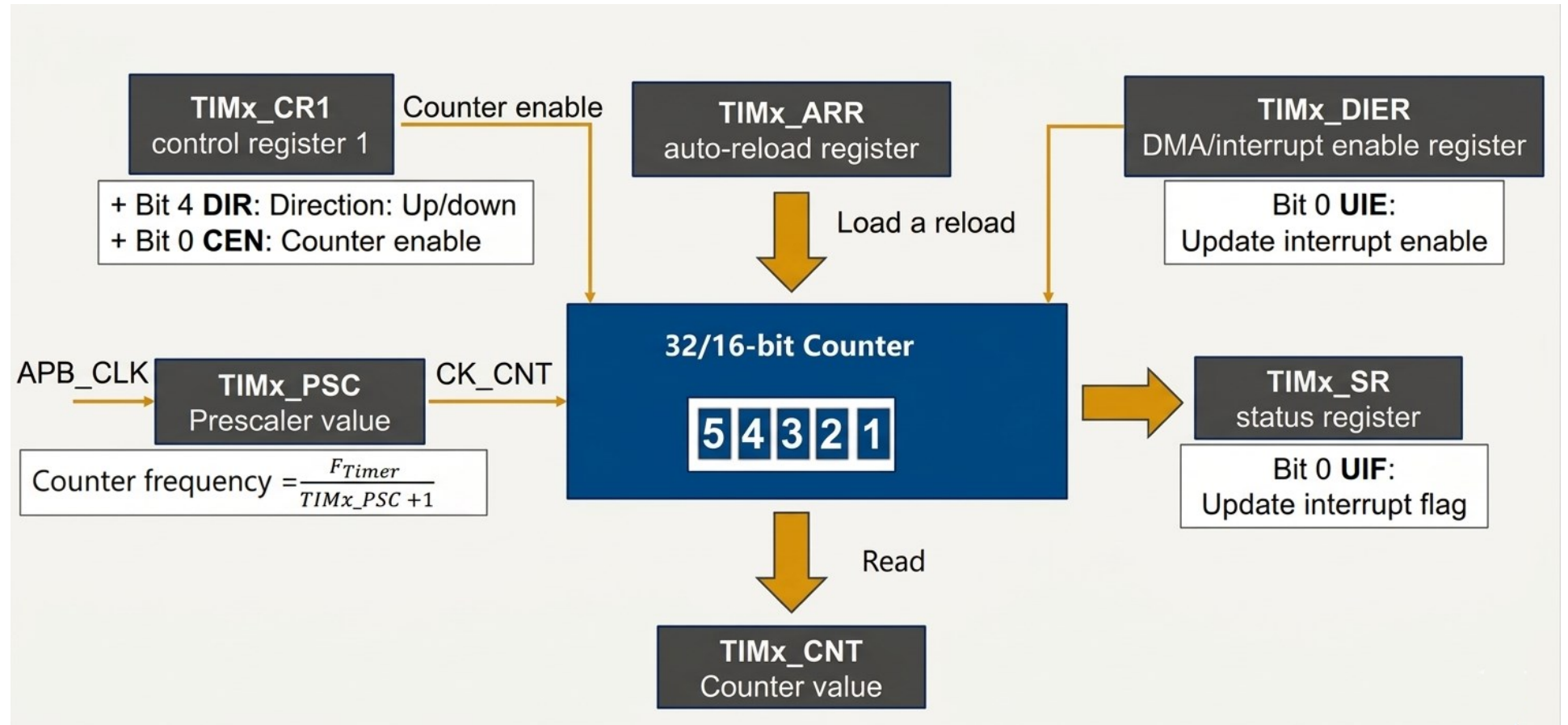
```
/* PC13 output */
GPIOC_CRH &= ~(0xF << 20); // clear
GPIOC_CRH |= (0x1 << 20); // set

while(1)
{
    GPIOC_ODR ^= (1<<13); // toggle LED
    delay();
}
}
```

Timer

Loại Timer	Tên	Số lượng	Độ rộng	Bus	Đặc điểm chính
Advanced-control	TIM1	1	16-bit	APB2	PWM nâng cao, dead-time, complementary
General-purpose	TIM2	1	32-bit	APB1	Counter dài, timing chính xác
General-purpose	TIM3, TIM4	2	16-bit	APB1	PWM, input capture, output compare
SysTick	SysTick	1	24-bit	Core	Timer hệ thống (ARM)

Timer



Cấu hình Timer

Bước 1: Cấp xung nhịp (Clock) cho Timer

Bước 2: Thiết lập đơn vị cơ sở thời gian (Time-base unit)

Bước 3: Chọn chế độ và hướng đếm (Counter Mode)

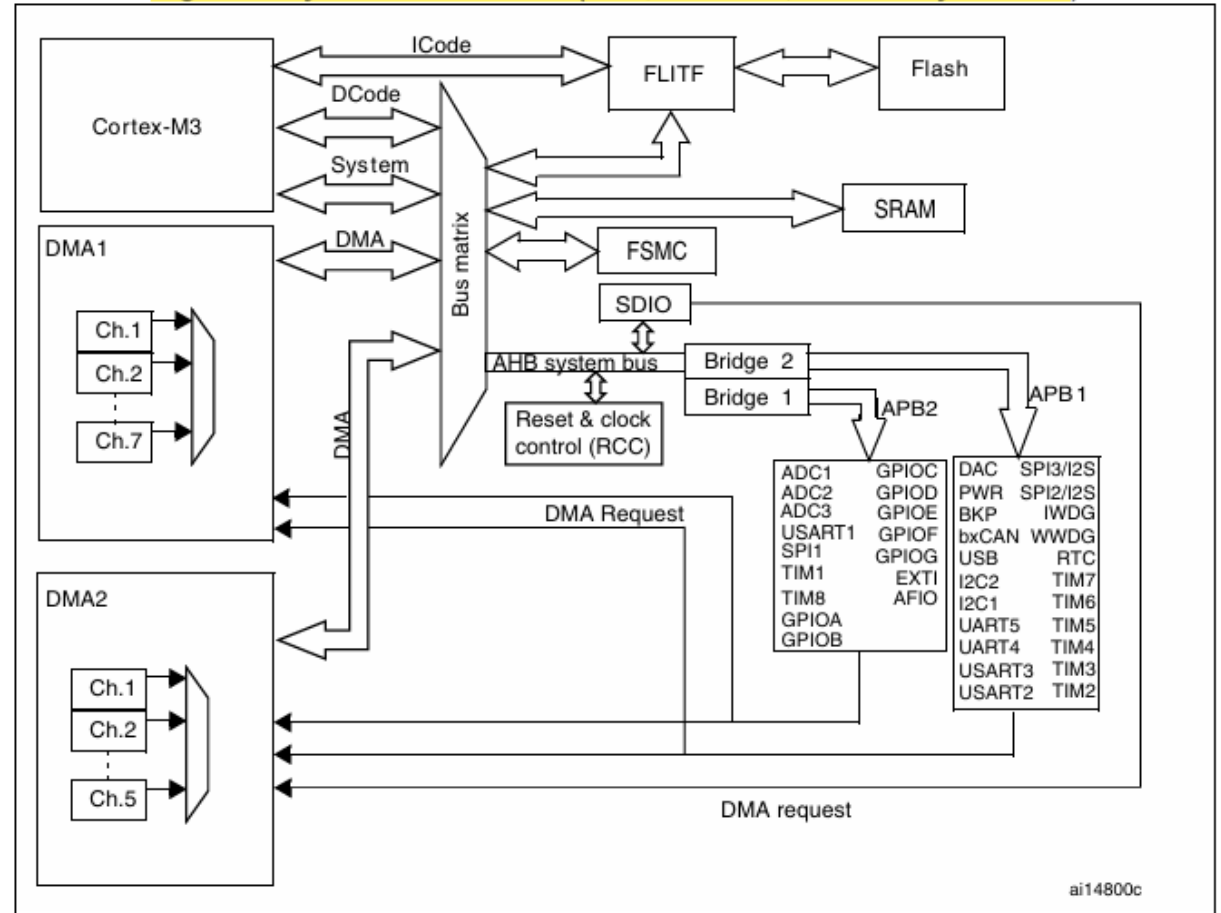
Bước 4: Khởi tạo sự kiện cập nhật (Update Event)

Bước 5: Cấu hình Ngắt hoặc DMA (Tùy chọn) và Kích hoạt Timer

Cấu hình Timer – Bước 1 (Cấp xung nhịp (Clock) cho Timer)

- Với TIM1: Nằm trên bus APB2, bật bằng cách thiết lập bit TIM1EN trong thanh ghi RCC_APB2ENR
- Với TIM2, TIM3, TIM4: Nằm trên bus APB1, bật bằng cách thiết lập các bit TIMxEN tương ứng trong thanh ghi RCC_APB1ENR

Figure 1. System architecture (low-, medium-, XL-density devices)

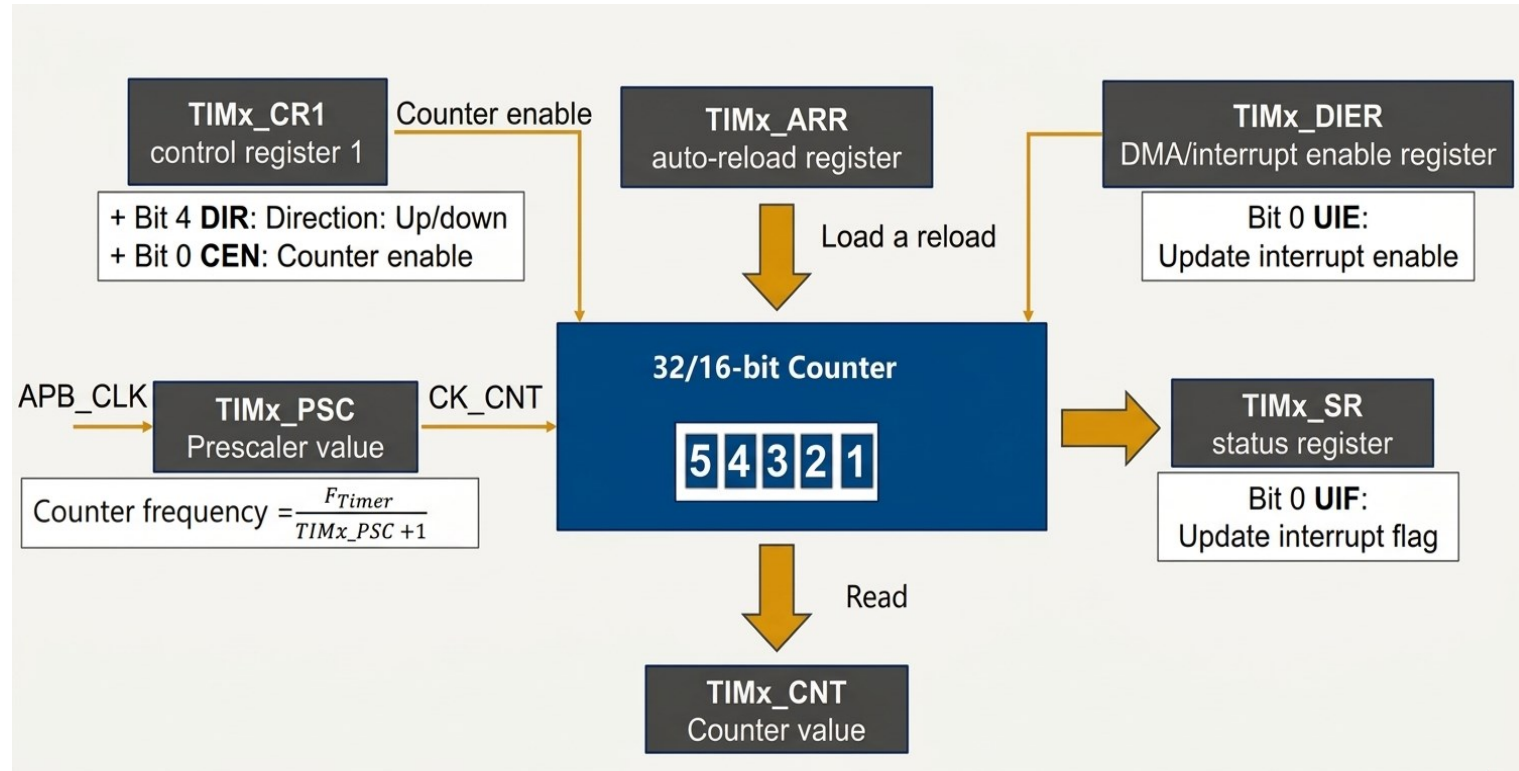


Cấu hình Timer – Bước 2 (Thiết lập đơn vị cơ sở thời gian)

- Công thức chung

$$T = \frac{(PSC + 1) \times (ARR + 1)}{F_{Timer}}$$

Thành phần	Ý nghĩa
PSC	chia tần
ARR	giá trị giới hạn
CNT	giá trị hiện tại
f_timer	clock đầu vào



Cấu hình Timer – Bước 3 (Chọn chế độ và hướng đếm)

- Hướng đếm (DIR bit):
 - **Đếm lên (0):** Đếm từ 0 đến ARR rồi quay lại 0
 - **Đếm xuống (1):** Đếm từ ARR về 0 rồi quay lại ARR
 - **Căn giữa (Center-aligned):** Đếm từ 0 đến ARR (overflow) rồi đếm lùi về 0 (underflow)
- Mỗi timer có 4 kênh độc lập:
 - **Input Capture:** Lấy giá trị bộ đếm khi có tín hiệu ngoại, dùng để đo tần số hoặc độ rộng xung
 - **Output Compare:** Thay đổi trạng thái chân ngõ ra khi bộ đếm đạt đến giá trị cài đặt
 - **PWM:** Tạo ra tín hiệu điều chế độ rộng xung với chu kỳ và độ rộng có thể lập trình
 - **One-pulse:** một xung

Cấu hình Timer – Bước 4 (Khởi tạo sự kiện cập nhật (Update Event))

- Để các cấu hình về bộ chia (PSC) và giá trị nạp lại (ARR) có hiệu lực ngay lập tức vào các thanh ghi, thiết lập bit UG trong thanh ghi **TIMx_EGR**, việc này sẽ reset bộ đếm về 0 và cập nhật các thông số đã cài đặt.

Các bước cấu hình ngắt Timer

- **Bước 1:** Thiết lập bit **UIE** (Update interrupt enable) trong thanh ghi **TIMx_DIER** để cho phép tạo ngắt khi có sự kiện cập nhật (như tràn bộ đếm)
- **Bước 2:** Xác định Vector ngắt - Mỗi timer có một vị trí trong bảng vector ngắt (ví dụ: TIM2 tại vị trí 28, TIM3 tại 29)
- **Bước 3:** Bật kênh ngắt - Sử dụng các thanh ghi của NVIC để kích hoạt kênh ngắt tương ứng với timer đang sử dụng
- **Bước 4:** Thiết lập ưu tiên (Tùy chọn) - Cấu hình mức độ ưu tiên cho ngắt này để tránh xung đột với các ngắt khác
- **Bước 5:** Viết trình phục vụ ngắt, với tên được tham khảo trong **startup_stm32f10x_md.s**

Cấu hình Timer – Bước 5 (Cấu hình Ngắt hoặc DMA (Tùy chọn) và Kích hoạt Timer)

- **Ngắt** : Bật bit **UIE** trong thanh ghi **TIMx_DIER** để kích hoạt ngắt khi có sự kiện cập nhật
- **DMA**: Bật bit **UDE** trong **TIMx_DIER** nếu muốn gửi yêu cầu DMA khi timer tràn
- Cuối cùng, thiết lập bit **CEN** (Counter Enable) trong thanh ghi **TIMx_CR1** để cho phép bộ đếm bắt đầu hoạt động

Cấu hình Timer – Ví dụ

```
#define RCC_CR    (*(volatile unsigned int*)0x40021000)
#define RCC_CFGR  (*(volatile unsigned int*)0x40021004)
#define RCC_APB2ENR (*(volatile unsigned int*)0x40021018)
#define RCC_APB1ENR (*(volatile unsigned int*)0x4002101C)

#define GPIOC_CRH  (*(volatile unsigned int*)0x40011004)
#define GPIOC_ODR  (*(volatile unsigned int*)0x4001100C)

#define TIM2_CR1    (*(volatile unsigned int*)0x40000000)
#define TIM2_DIER   (*(volatile unsigned int*)0x4000000C)
#define TIM2_SR     (*(volatile unsigned int*)0x40000010)
#define TIM2_CNT    (*(volatile unsigned int*)0x40000024)
#define TIM2_PSC    (*(volatile unsigned int*)0x40000028)
```

Cấu hình Timer – Ví dụ

```
#define TIM2_ARR    (*(volatile unsigned int*)0x4000002C)
```

```
#define NVIC_ISERO  (*(volatile unsigned int*)0xE000E100)
```

```
void TIM2_IRQHandler(void)
{
    if (TIM2_SR & 1)    // check UIF
    {
        TIM2_SR &= ~1;    // clear flag
        GPIOC_ODR ^= (1<<13); // toggle LED
    }
}
```

Cấu hình Timer – Ví dụ

```
int main()
{
    RCC_CR |= (1<<16);          // HSE ON
    while(!(RCC_CR & (1<<17)));  // Wait HSE ready

    RCC_CFGR &= ~(3<<0);
    RCC_CFGR |= (1<<0);          // SYSCLK = HSE

    RCC_APB2ENR |= (1<<4); // GPIOC
    RCC_APB1ENR |= (1<<0); // TIM2

    GPIOC_CRH &= ~(0xF << 20);
    GPIOC_CRH |= (0x2 << 20);
```

```
TIM2_PSC = 8000 - 1;
TIM2_ARR = 50;

TIM2_CR1 &= ~(1<<4);
TIM2_CR1 |= (1<<7);

TIM2_CNT = 0;

TIM2_DIER |= 1;
NVIC_ISER0 |= (1<<28);

TIM2_CR1 |= 1;

while(1);
}
```

Bài tập

Yêu cầu chung:

Thực hiện các bài tập dưới đây bằng cách thiết kế và mô phỏng trên Proteus sử dụng vi điều khiển STM32F103C8. Đối với mỗi bài, sinh viên cần:

- Trình bày nguyên lý hoạt động của hệ thống.
- Giải thích kết quả mô phỏng đạt được.
- Thực hiện cấu hình trực tiếp thông qua thanh ghi (register-level programming), không sử dụng các thư viện có sẵn (HAL, StdPeriph, v.v.).

Bài tập

Bài 1: kế mạch mô phỏng gồm vi điều khiển STM32F103C8 và hai LED đơn (LED1 và LED2). Yêu cầu:

- LED1 nhấp nháy với chu kỳ 100 ms.
- LED2 nhấp nháy với chu kỳ 1 giây.
- Sử dụng Timer để tạo độ trễ chính xác.

Bài 2: Thiết kế mạch mô phỏng gồm STM32F103C8, một nút nhấn và một LED (LED1). Yêu cầu:

- Đếm số lần nút được nhấn.
- Khi số lần nhấn đạt 10, LED1 sẽ đảo trạng thái (toggle).
- Sau khi đạt ngưỡng, có thể tiếp tục đếm lại từ đầu.

Bài tập

Bài 3: Xây dựng mô hình mô phỏng sử dụng servo làm kim đồng hồ. Yêu cầu:

- Cứ mỗi 1 giây, servo quay một góc khoảng 6° , tương ứng với chuyển động của kim giây.
- Sau 60 giây (tương ứng một chu kỳ hoàn chỉnh hoặc đạt giới hạn góc quay 180°), servo phải quay nhanh về vị trí ban đầu (0°).
- Chu kỳ hoạt động sau đó được lặp lại liên tục.
- Sử dụng PWM từ Timer để điều khiển servo, không sử dụng delay bằng loop.

Hết.